

Министерство Образования Республики Беларусь
Учреждение Образования
“Брестский Государственный Технический Университет”
Факультет электронно-информационных систем
Кафедра интеллектуально информационных технологий

Отчёт по лабораторной работе № 2
По дисциплине «Системное программирование»

Выполнил:
Тютюков К. О.
Студент группы ПО-13
Проверил:
А.А. Крощенко
Доцент кафедры ИИТ
02.03.2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python.

Ход работы:

Задание 1:

Множество вещественных чисел ограниченной мощности – Предусмотреть возможность пересечения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Мощность множества задается при создании объекта. Реализацию множества осуществить на базе списка. Переопределить метод `__eq__`, выполняющий сравнение объектов данного типа.

```
class LimitedSet:
    def __init__(self, max_power, elements=None):
        self.max_power = max_power
        self.items = []
        if elements:
            for e in elements:
                self.add(e)

    def add(self, value):
        if value not in self.items and len(self.items) < self.max_power:
            self.items.append(float(value))
            return True
        return False

    def remove(self, value):
        if value in self.items:
            self.items.remove(value)
            return True
        return False

    def contains(self, value):
        return value in self.items

    def intersection(self, other):
        result = LimitedSet(min(self.max_power, other.max_power))
        for item in self.items:
            if item in other.items:
                result.add(item)
        return result
```

```

def __str__(self):
    return "{" + ", ".join(str(x) for x in self.items) + "}"

def __eq__(self, other):
    return set(self.items) == set(other.items)

# Создание множеств
s1 = LimitedSet(5, [1.5, 2.7, 3.2, 4.8])
s2 = LimitedSet(5, [2.7, 4.8, 6.1, 7.5])
print("Множество 1:", s1)
print("Множество 2:", s2)
# Проверка принадлежности
print(f"Содержит 2.7? {s1.contains(2.7)}")
# Добавление элемента
s1.add(5.9)
print("После добавления 5.9:", s1)
# Удаление элемента
s1.remove(1.5)
print("После удаления 1.5:", s1)
# Пересечение
s3 = s1.intersection(s2)
print("Пересечение:", s3)
# Сравнение
s4 = LimitedSet(5, [2.7, 4.8])
print(f"Равны? {s3 == s4}")

```

```

Множество 1: {1.5, 2.7, 3.2, 4.8}
Множество 2: {2.7, 4.8, 6.1, 7.5}
Содержит 2.7? True
После добавления 5.9: {1.5, 2.7, 3.2, 4.8, 5.9}
После удаления 1.5: {2.7, 3.2, 4.8, 5.9}
Пересечение: {2.7, 4.8}
Равны? True

```

Задание 2:

Система Телефонная станция. Абонент оплачивает Счет за разговоры и Услуги, может попросить Администратора сменить номер и отказаться от услуг. Администратор изменяет номер, Услуги и временно отключает Абонента за неуплату.

```

class Subscriber:
    def __init__(self, name, phone_number):
        self.name = name
        self.phone_number = phone_number
        self.debt = 0
        self.services = []

```

```
self.is_active = True
```

```
def pay(self, amount):  
    if amount <= self.debt:  
        self.debt -= amount  
        print(f"{self.name} оплатил {amount} руб. Остаток: {self.debt}")  
    else:  
        print(f"Оплачено {amount}, сдача {amount - self.debt}")  
        self.debt = 0
```

```
def make_call(self, minutes):  
    cost = minutes * 0.2  
    self.debt += cost  
    print(f"Звонок {minutes} мин = {cost} руб.")
```

```
def add_service(self, service_name, service_price):  
    self.services.append(service_name)  
    self.debt += service_price  
    print(f"Добавлена услуга {service_name} ({service_price} руб.)")
```

```
def remove_service(self, service_name):  
    for s in self.services:  
        if s == service_name:  
            self.services.remove(s)  
            print(f"Услуга {service_name} удалена")  
            return  
    print(f"Услуга {service_name} не найдена")
```

```
class Administrator:  
    def __init__(self, name):  
        self.name = name
```

```
def change_number(self, subscriber, new_number):  
    old = subscriber.phone_number  
    subscriber.phone_number = new_number  
    print(f"{self.name}: сменил номер {old} -> {new_number}")
```

```
def disable(self, subscriber):  
    if subscriber.debt > 0:  
        subscriber.is_active = False  
        print(f"{self.name}: отключил {subscriber.name} (долг {subscriber.debt} руб.)")
```

```
def enable(self, subscriber):  
    if subscriber.debt == 0:  
        subscriber.is_active = True
```

```
print(f"{self.name}: включил {subscriber.name}")
```

```
admin = Administrator("Иван")
sub = Subscriber("Петр", "123-45-67")
print(f"Абонент: {sub.name}, Номер: {sub.phone_number}\n")
sub.make_call(10)
sub.add_service("Интернет", 30)
sub.add_service("ТВ", 20)
print(f"\nДолг: {sub.debt} руб.")
print(f"Услуги: {sub.services}\n")
sub.pay(50)
print(f"Остаток долга: {sub.debt}\n")
admin.change_number(sub, "999-88-77")
admin.disable(sub)
print(f"Активен: {sub.is_active}")
```

```
Абонент: Петр, Номер: 123-45-67

Звонок 10 мин = 2.0 руб.
Добавлена услуга Интернет (30 руб.)
Добавлена услуга ТВ (20 руб.)

Долг: 52.0 руб.
Услуги: ['Интернет', 'ТВ']

Петр оплатил 50 руб. Остаток: 2.0
Остаток долга: 2.0

Иван: сменил номер 123-45-67 -> 999-88-77
Иван: отключил Петр (долг 2.0 руб.)
Активен: False
```

Вывод работы: закрепил навыки объектно-ориентированного программирования на языке Python.